

```
=====
                        ARRAY ADDRESSING PROGRAM
                        in C Language
=====
```

```
PROGRAM DESCRIPTION:
```

This program demonstrates array addressing and pointer usage to display the address of array elements. It shows how to access memory addresses of array elements using the address-of operator (&).

```
=====
                        SOURCE CODE
=====
```

```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int i;
    int *ptr;
    printf("Enter the index: ");
    scanf("%d", &i);

    printf("Address of arr[%d] = %p\n", i, &arr[i]);

    return 0;
}
```

```
=====
                        PROGRAM EXPLANATION
=====
```

1. ARRAY DECLARATION:
int arr[5] = {10, 20, 30, 40, 50};
 - Creates an integer array with 5 elements
 - Initializes with values 10, 20, 30, 40, 50
 - Array indices: 0, 1, 2, 3, 4
2. VARIABLE DECLARATIONS:
 - i: Integer variable to store the index
 - ptr: Pointer to integer (for potential pointer operations)
3. INPUT:
 - User enters an index value (0-4)
4. OUTPUT:
 - Displays the memory address of the element at the given index
 - Format: "Address of arr[index] = 0x[hex_address]"
 - Uses %p format specifier for printing addresses
 - &arr[i] gives the address of element at index i

```
=====
                        HOW TO COMPILE & RUN
=====
```

Compilation:
gcc -o array_addressing array_addressing.c

Execution:
./array_addressing

Example Input & Output:

Enter the index: 2

Address of arr[2] = 0x7ffc8b4a4f48

(Note: Actual addresses will vary based on system and execution)

KEY CONCEPTS

1. ADDRESS-OF OPERATOR (&):

- & is the address-of operator
- &arr[i] returns the memory address of arr[i]
- Not the value, but where the value is stored in memory

2. POINTER BASICS:

- int *ptr declares a pointer to an integer
- Pointers store memory addresses
- Can be used to access and manipulate data indirectly

3. ARRAY IN MEMORY:

For array [10, 20, 30, 40, 50]:

Index	Value	Address (example)
-----	-----	-----
0	10	0x7ffc8b4a4f3c
1	20	0x7ffc8b4a4f40
2	30	0x7ffc8b4a4f44
3	40	0x7ffc8b4a4f48
4	50	0x7ffc8b4a4f4c

(Addresses increment by 4 bytes for int, typically)

4. FORMAT SPECIFIERS:

- %d: Integer value
- %p: Pointer/Address (prints in hexadecimal)
- %x: Hexadecimal number

ARRAY ADDRESSING DETAILS

Memory Layout:

- Arrays are stored in contiguous memory locations
- Each int typically occupies 4 bytes
- Elements are stored sequentially in memory

Address Calculation:

- Address of arr[i] = Base Address + (i * sizeof(int))
- Base Address = Address of arr[0]

Pointer Arithmetic:

- ptr = &arr[0] makes ptr point to first element
- ptr + 1 points to second element
- ptr + i points to (i+1)th element
- *(ptr + i) gives value of arr[i]

PRACTICAL APPLICATIONS

1. DYNAMIC MEMORY ACCESS:

- Accessing array elements through pointers
- Useful in advanced data structure implementations

2. FUNCTION PARAMETERS:

- Pass array elements by reference using pointers
- Modify array elements in functions

3. DATA STRUCTURE CREATION:

- Foundation for linked lists, trees, graphs
- Pointer-based data management

4. MEMORY DEBUGGING:

- Understanding memory layout
- Debugging memory-related issues

```
=====
COMMON PATTERNS
=====
```

Pattern 1: Direct Array Access

```
printf("%d\n", arr[i]);    // Access value directly
```

Pattern 2: Pointer-based Access

```
printf("%d\n", *(ptr + i)); // Access value through pointer
```

Pattern 3: Address Access

```
printf("%p\n", &arr[i]);   // Print address
```

Pattern 4: Store Address in Pointer

```
int *p = &arr[i];
printf("%d\n", *p);        // Dereference pointer to get value
=====
```